

HW#5 Domain-Specific Accelerator

鄭維, 10801069

Abstract—This work explores the integration of hardware and software optimizations to enhance the performance of a convolutional neural network (CNN) model. By implementing a domain-specific accelerator (DSA), tightly coupled memory (TCM), and software modifications, I aim to address computational and memory bottlenecks. Results demonstrate significant speedups from DSA acceleration, with smaller contributions from padding and stride optimizations and TCM usage. While the combined effects are additive rather than multiplicative, the study highlights the critical role of computational efficiency. Future improvements are proposed to further optimize memory access, parallelize computations, and streamline software management.

I. INTRODUCTION

This study integrates hardware and software optimizations to enhance a convolutional neural network (CNN) for handwritten character recognition. A domain-specific accelerator (DSA) is utilized for fused multiply-add operations, supported by tightly coupled memory (TCM) to enable efficient data storage and retrieval. Software enhancements, including hardcoding padding and stride values, further streamline computational processes.

The objective is to identify and mitigate bottlenecks in computation and data feeding, achieving substantial performance improvements. This work evaluates the individual and combined impacts of these optimizations, offering insights into their effectiveness and interplay. By focusing on practical solutions and analyzing their outcomes, this study provides a foundation for further exploration of hardware-software co-optimization in convolutional neural network processing.

II. METHODOLOGY

The analysis of the platform integrates hardware and software components to enhance the performance of a convolutional neural network (CNN) model. The methodology focuses on leveraging domain-specific accelerators (DSA), tightly coupled memory (TCM), and software optimizations that fully utilize the hardware's capabilities for efficient computations and data handling.

A. Hardware

A domain-specific accelerator (DSA) is designed to execute fused multiply-add operations. Using the Xilinx floating-point IP, the DSA calculates results in the form $(A \times B) + C$. Computations are initiated by storing data at specific memory-mapped addresses. The results are wired back to facilitate iterative dot-product computations, which are essential in CNN operations. Complementing the DSA is a 128 KB tightly coupled memory (TCM), implemented using block RAM. This memory is mapped to a dedicated address space, enabling high-speed access to data such as CNN weights and feature maps.

B. Profiler

An embedded profiler within the DSA module tracks execution cycles to assess performance. It measures computation time as the cycle count from when all input data is fed into the DSA to the generation of a valid output, while data feeding time is calculated as the remaining portion of the total cycles.

C. Software

Padding and stride parameters in convolutional layers are hardcoded to fixed values (0 and 1, respectively), reducing overhead from variable declarations, constraint checks, and redundant operations. CNN weights and feature maps are stored in the TCM, enabling quick access and minimizing memory latency. The DSA accelerates vector dot product operations in the **conv_3d** and **fully_connected_layer_forward_propagation** functions, enhancing computational performance. Various configurations are evaluated, including baseline performance, padding and stride optimization, DSA-only acceleration, TCM-only optimization, and a fully optimized setup that combines all enhancements.

III. RESULTS

The speedup is computed according to the following formula:

$$Speedup = \frac{T}{T'}$$

Here, T represents performance (ms), with a prime symbol (') indicating the improved value.

TABLE I. CONFIGURATION COMPARISON

Configuration	Performance	
	Time (ms)	Speedup (x times)
Baseline	25482.6	1.00
Optimized padding and stride	21025.4	1.21
TCM optimization	20895.6	1.22
DSA optimization	3431.0	7.43
Full optimization	3278.0	7.77

Since the profiler tracks execution cycles for the DSA, only optimization with DSA would be profiled.

TABLE II. DSA ANALYSIS

Configuration	Proportion	
	Data feeding (%)	Computing (%)
DSA optimization	30.80	69.20
Full optimization	29.74	70.26

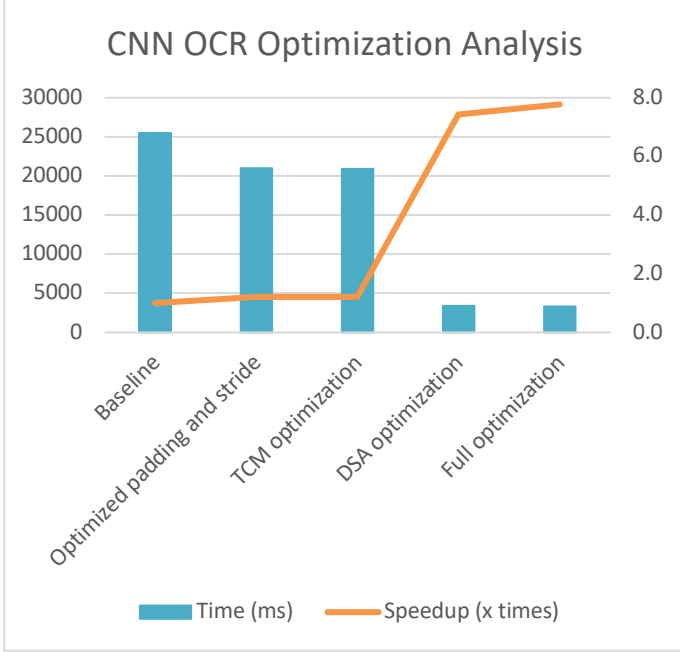


Fig. 1. CNN OCR Optimization Analysis

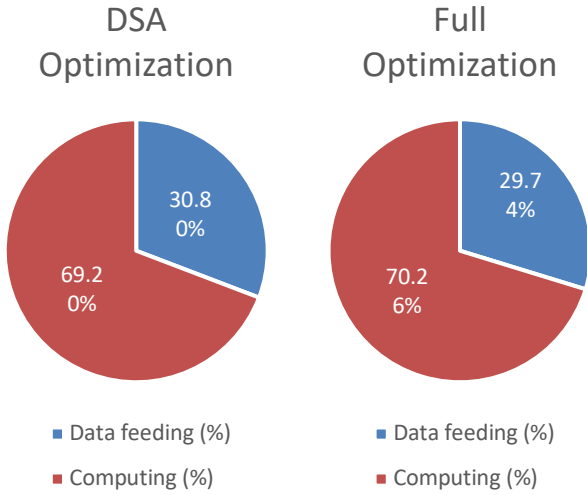


Fig. 2. DSA Analysis

IV. DISCUSSION

Optimizing padding and stride by modifying the C code achieved a 1.21x speedup. Similarly, moving weights and feature maps from main memory with data caching to tightly coupled memory (TCM) provided comparable performance improvements. Accelerating vector dot product computations using the DSA resulted in a significant speedup, outperforming the previous optimizations. However, combining these optimizations only led to a slight improvement compared to DSA optimization alone.

The combined speedup is not multiplicative, likely because these optimizations target different aspects of the program and their effects are additive rather than synergistic. Since DSA optimization dominates the overall speedup, the contributions of other enhancements appear minimal.

Table II. and Fig. 2 indicates that the proportion of time spent on data feeding during DSA optimization and full optimization is similar and remains lower than the time spent on computation. While there is room for further improvement in data feeding, the computational overhead highlights its critical role in overall optimization.

V. CONCLUSION

This study demonstrates the integration of hardware and software optimizations to enhance CNN performance, leveraging domain-specific accelerators, tightly coupled memory, and tailored software modifications. DSA significantly improves computational efficiency, dominating the performance gains, while additional optimizations such as padding and stride adjustments, and the use of TCM, provide smaller but complementary enhancements. However, the combined effect of these optimizations is additive rather than multiplicative, as they target different bottlenecks in the execution pipeline. The analysis also highlights the importance of balancing computational and data feeding efficiencies, with computation remaining the dominant overhead.

Future improvements could focus on optimizing memory access by better utilizing the AXI stream bus, adding larger data feeding buffers, and expanding data port widths to enable direct memory access for the DSA. Parallelizing computations by incorporating additional floating-point IPs for vector dot product operations could further enhance computational throughput. On the software side, eliminating redundant variable management, such as recalculating constants that remain unchanged across channels, could reduce overhead and improve efficiency.